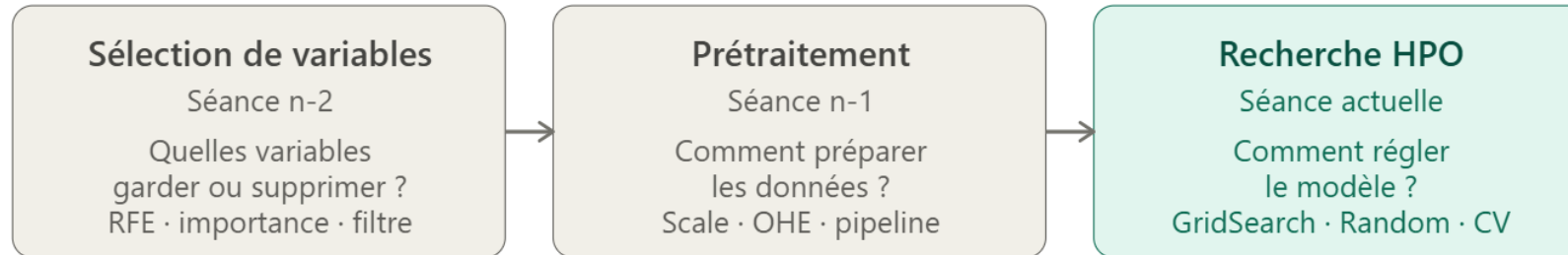




GRANDE ÉCOLE D'INGÉNIEURS

" Changez le monde : devenez ingénieurs "

Trois séances, trois questions fondamentalement différentes



Ce qui est fixé vs ce qui varie dans chaque séance

Sélection : modèle fixé, on cherche le bon sous-ensemble de colonnes

Prétraitement : variables fixées, on cherche la bonne transformation (scale, OHE, imputation)

HPO : données + pipeline fixés, on cherche les meilleurs réglages du modèle (C, k, max_depth...)
On peut aussi optimiser les étapes du pipeline (strategy='median' vs 'mean', RobustScaler vs StandardScaler)

Analogie culinaire

Sélection = choisir
les ingrédients

Prétraitement = préparer
les ingrédients (couper, cuire)

HPO = régler le four
(température, durée, position)

Les 3 méthodes

Séance n-2

Sélection de variables

**Quelles colonnes
garder ou supprimer ?**

Ce qui est fixé :

Modèle fixé • Prétraitement fixé

Ce qu'on cherche :

Le meilleur sous-ensemble de features

RFE • importance RF • corrélation • filtre

Séance n-1

Prétraitement

**Comment transformer
les données ?**

Ce qui est fixé :

Variables fixées • Modèle fixé

Ce qu'on cherche :

La bonne transformation (scale, OHE...)

StandardScaler • OneHotEncoder • Pipeline

Aujourd'hui

Recherche HPO

**Comment régler
le modèle ?**

Ce qui est fixé :

Données fixées • Pipeline fixé

Ce qu'on cherche :

Les meilleurs réglages du modèle

GridSearchCV • RandomizedSearch • CV imbriquée

L'analogie du chef cuisinier



Sélection des variables

Choisir les ingrédients

Quels ingrédients mettre dans la recette ? On élimine ce qui n'apporte rien (variables inutiles, redondantes) et on garde ce qui compte vraiment.

Exemple : supprimer les colonnes de bruit, garder les features importantes (RFE).



Prétraitement

Préparer les ingrédients

Comment couper, cuire, assaisonner les ingrédients ? On transforme les données pour les rendre exploitables par le modèle.

Exemple : normaliser (StandardScaler), encoder (OneHotEncoder), imputer les NaN.



Hyperparamètres

Régler le four

À quelle température ? Pendant combien de temps ? Avec quelle chaleur ? On règle les paramètres de l'algorithme d'apprentissage.

Exemple : trouver $C=10$, $\gamma=0.01$ pour SVM via GridSearchCV.

Le concept central de cette séance

Paramètre vs Hyperparamètre

PARAMÈTRE — appris automatiquement

Définition	Valeur calculée par l'algorithme pendant l'entraînement, en minimisant une fonction de coût.
Qui le choisit ?	Personne ; le modèle l'apprend seul depuis les données.
Exemples	• Poids w d'une régression linéaire • Vecteurs support d'un SVM • Seuils de coupure d'un arbre • Centres des clusters k-means
Analogie	La recette que le cuisinier apprend avec l'expérience.

VS

HYPERPARAMÈTRE — choisi par l'ingénieur

Définition	Réglage de l'algorithme fixé AVANT l'entraînement. Il contrôle comment le modèle apprend.
Qui le choisit ?	L'ingénieur — c'est exactement l'objet de cette séance !
Exemples	• C et γ du SVM • k de KNN • α de Ridge/Lasso • $n_estimators$, max_depth de RF
Analogie	La température du four : le chef la règle, elle n'est pas dans la recette.

A comprendre avant de commencer...

Leakage HPO : quand la recherche triche sans le savoir

✗ Approche naïve (à ne pas faire)

1. Diviser en train / test
2. Tester $C = 0.1, 1, 10, 100...$ sur les données de TEST
3. Choisir le meilleur C selon le score de TEST
4. Annoncer ce score comme performance du modèle



Le jeu de test a servi à prendre une décision → il n'est plus « hors sample » → score TROP OPTIMISTE

✓ GridSearchCV (approche correcte)

1. Garder un jeu de test totalement isolé
 2. Sur le TRAIN : GridSearchCV avec CV Interne (5 folds)
 3. GridSearch choisit les meilleurs hyperparamètres SANS toucher au test
 4. Évaluer une seule fois sur le test à la fin
- ✓ Le test reste « hors sample » → score honnête → performances réelles en production

Les 3 méthodes que vous allez utiliser

1 GridSearchCV

Exhaustif

Teste TOUTES les combinaisons de la grille définie.

Quand l'utiliser :

Quand utiliser : espace petit (< 100 combinaisons), chaque entraînement rapide.

+ Garantit le meilleur résultat dans la grille.

- Coûteux : 3 valeurs × 4 valeurs × 5 folds = 60 entraînements.

```
GridSearchCV(pipe,  
param_grid={'C':[0.1,1,10]},  
cv=5, scoring='accuracy')
```

2 RandomizedSearchCV

Aléatoire

Tire n_iter combinaisons aléatoires depuis des distributions.

Quand l'utiliser :

Quand utiliser : grand espace (RF, XGBoost), distributions continues pour C, alpha.

+ Trouve souvent = même qualité avec 10× moins d'évaluations.

- Pas garanti d'être optimal, mais suffisant en pratique.

```
RandomizedSearchCV(pipe,  
param_dist={'C':  
loguniform(0.001,1000)},  
n_iter=30, cv=5)
```

3 Nested CV

Honnête

CV externe (évaluation) × GridSearch interne (sélection HPO).

Quand l'utiliser :

Quand utiliser : pour estimer honnêtement les perfs du pipeline HPO complet.

+ Évite le leakage HPO → score non biaisé, proche de la réalité production.

- Plus lent : (5 folds externes × 3 folds internes × n combos) entraînements.

```
cross_val_score(  
GridSearchCV(pipe, grid, cv=3),  
X, y, cv=5)
```

Exemple concret : différence entre sélection de variables, pré-traitement et recherche

Objectif : prédire si un client va acheter un produit

Exemples de variables disponibles :

- âge
- salaire
- ville
- historique d'achat
- nombre de visites

Sélection de variables

Définition :

Choisir les variables pertinentes pour le modèle

Exemple :

Variables initiales :

âge, salaire, ville, code_postal, historique_achat, nb_visites

Après sélection :

âge, salaire, historique_achat, nb_visites

Objectifs :

- réduire le bruit
- améliorer les performances
- limiter le surapprentissage

Pré-traitement des données

Définition :

Transformer les données pour les rendre exploitables

Exemples :

- Normalisation des variables numériques
- Encodage des variables catégorielles
- Gestion des valeurs manquantes

Exemples concrets :

- âge → standardisation
- ville → encodage one-hot
- salaire manquant → remplacement par la moyenne

Recherche d'hyperparamètres

Définition :

Optimiser les paramètres du modèle

Exemple avec Random Forest :

- `n_estimators` : nombre d'arbres
- `max_depth` : profondeur maximale
- `min_samples_split` : taille minimale pour diviser

Objectif :

Trouver la combinaison qui maximise la performance

Principe de GridSearch

Méthode :

Tester toutes les combinaisons possibles d'hyperparamètres

Exemple de grille :

- n_estimators : 50, 100, 200
- max_depth : 5, 10, None
- min_samples_split : 2, 5, 10

Résultat :

Le modèle avec la meilleure performance est sélectionné

Nombre total de combinaisons :

$3 \times 3 \times 3 = 27$ modèles testés

Pour chaque combinaison d'hyperparamètres :

- 1.Le modèle est entraîné sur les données d'apprentissage
- 2.Une validation croisée est effectuée
- 3.Un score de performance est calculé

Ce processus est répété pour toutes les combinaisons

4 groupes — 4 modèles cibles différents

Groupe 1 (4)

SVM sur Breast Cancer
C · gamma · kernel
surface 2D (C × gamma)
pipeline + HPO

Groupe 2 (4)

KNN sur Digits
k · metric · weights
malédiction de la dim.
n_components (PCA)

Groupe 3 (3)

Random Forest · Housing
n_estimators · max_depth
min_samples · max_features
RandomizedSearch

Groupe 4 (2)

Ridge/Lasso · Diabetes
alpha · l1_ratio
ElasticNet · LassoCV
chemin régularisation



Merci pour votre attention



eSEO

eSEO

GRANDE ÉCOLE D'INGÉNIEURS
Angers • Dijon • Paris-Vélizy